

ОТЧЁТ О РАЗРАБОТКЕ ПРОГРАММНОГО КОМПЛЕКСА

Тема: Разработка Telegram-бота для мониторинга товаров на торговой площадке Funpay с интеграцией СУБД SQLite3 и парсера на BeautifulSoup4

1. ВВЕДЕНИЕ

1.1 Актуальность темы

Современный рынок игровых услуг и виртуальных товаров характеризуется высокой динамичностью и огромным объемом информации. Торговые площадки, такие как FunPay, ежедневно обрабатывают тысячи объявлений в сотнях различных игровых категорий. Для конечного пользователя ручной поиск наиболее выгодных предложений, сравнение цен и отслеживание обновлений сопряжены с большими временными затратами.

Автоматизация данного процесса с использованием интерфейса Telegram-ботов является актуальным решением. Это позволяет перенести сложную логику парсинга и фильтрации данных на сторону сервера, предоставляя пользователю лаконичный, быстрый и интуитивно понятный инструмент для получения информации в режиме реального времени.

1.2 Цель и задачи проекта

Цель работы: Проектирование, разработка и развертывание асинхронного программного комплекса (Telegram-бота), интегрированного с локальной базой данных и модулем веб-скрейпинга, для оперативного предоставления пользователю топ-5 актуальных товаров по выбранной игровой дисциплине.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить структуру HTML-кода целевой торговой площадки и разработать модуль первичного сбора данных (каталога игр и подкатегорий).
 2. Спроектировать реляционную структуру базы данных для эффективного хранения и индексации игровых категорий.
 3. Реализовать скрипты для миграции и валидации данных из промежуточных форматов (Excel) в СУБД.
 4. Разработать архитектуру Telegram-бота на базе современного асинхронного фреймворка aiogram 3.x.
 5. Реализовать алгоритм гибкого (нечеткого) поиска игр по текстовому запросу пользователя.
 6. Написать эффективный парсер страниц товаров, оптимизированный под обход ограничений и извлечение целевых узлов (динамических цен, названий и ссылок).
-

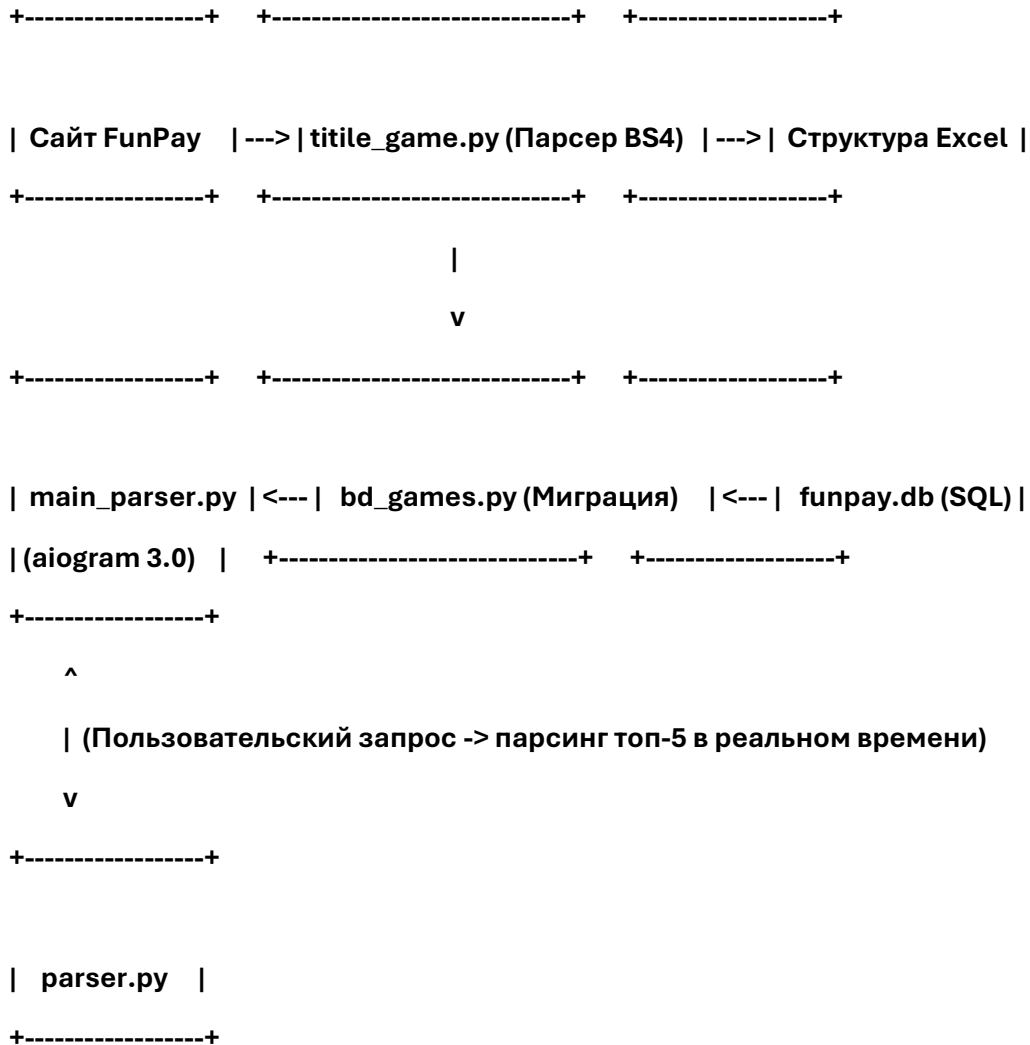
2. АНАЛИЗ И ОБОСНОВАНИЕ ВЫБОРА СТЕКА ТЕХНОЛОГИЙ

Эффективность программного комплекса напрямую зависит от выбранного инструментария. В рамках данного проекта был сформирован следующий технологический стек:

- **Язык программирования Python 3.10+:** Выбран благодаря высокой скорости разработки, развитой экосистеме библиотек для анализа данных и встроенной поддержке асинхронности (asyncio).
 - **Фреймворк aiogram 3.x:** В отличие от устаревшего и синхронного Telebot, aiogram построен на базе asyncio. Это критически важно для сетевых роботов, так как асинхронная архитектура позволяет обрабатывать тысячи запросов пользователей параллельно в одном потоке, не блокируя работу программы во время ожидания ответов от Telegram API или сервера FunPay.
 - **СУБД SQLite3:** Легковесная, встраиваемая реляционная база данных. Она не требует развертывания отдельного сервера базы данных, хранит все данные в одном файле (funpay.db), при этом полностью поддерживает стандарт SQL, индексы и транзакции, обеспечивая мгновенный отклик при локальных поисковых запросах.
 - **BeautifulSoup4 и Requests:** Классическая связка для веб-скрейпинга. BeautifulSoup4 преобразует сырой HTML-код страницы в дерево объектов Python, позволяя быстро находить нужные теги по классам и атрибутам.
 - **Библиотека OpenPyXL / Pandas:** Использовалась для работы с файлами формата .xlsx на этапе структурирования первичного каталога.
 - **Библиотека Python-dotenv:** Применяется для реализации паттерна «Twelve-Factor App» в части конфигурации. Позволяет хранить конфиденциальные данные (токен бота) в изолированном файле .env, исключая их утечку в публичный репозиторий.
-

3. АРХИТЕКТУРА И ЭТАПЫ РАЗРАБОТКИ ПРОГРАММНОГО КОМПЛЕКСА

Разработка системы велась поэтапно, разделяя логику сбора данных, их хранения и пользовательского интерфейса.



Этап 1. Разработка парсера каталога и структурирование данных

На первом этапе необходимо было получить полную карту сайта FunPay (все доступные игры и направления).

- За данную задачу отвечает скрипт `titile_game.py`.
- С помощью библиотеки `requests` отправляется GET-запрос к главной странице площадки.
- `BeautifulSoup4` производит поиск элементов, содержащих ссылки на игровые разделы и текстовые маркеры категорий.
- Собранные данные упорядочиваются и записываются в файл `funpay_games_structure.xlsx` с помощью библиотек работы с таблицами. Данный шаг необходим для промежуточной верификации данных разработчиком.

Этап 2. Проектирование базы данных и миграция

- Скрипт `bd_games.py` инициализирует базу данных `funpay.db`.

- Создается реляционная таблица для хранения игр со следующими полями: id (первичный ключ), game_name (название игры), url (ссылка на раздел) и category (тип товара, например: аккаунты, золото, услуги).
- Скрипт последовательно считывает строки из funpay_games_structure.xlsx и через параметризованные SQL-запросы (INSERT INTO ...) заполняет таблицы СУБД.

Этап 3. Разработка асинхронного Telegram-бота

Модуль main_parser.py является ядром системы и связывает базу данных, внешние скрипты парсинга и интерфейс пользователя.

Реализованный алгоритм взаимодействия:

1. Команда /start: Инициализирует приветственное сообщение и переводит пользователя в состояние ожидания текстового запроса.
2. Обработка ввода и нечеткий поиск: Когда пользователь вводит строку (например, pubg), бот выполняет асинхронный SQL-запрос к funpay.db с использованием конструкции LIKE '%pubg%'. Это позволяет найти все вхождения. Бот динамически генерирует клавиатуру с типом InlineKeyboardMarkup, где каждая кнопка соответствует найденной игре (например: PUBG и PUBG Mobile).
3. Выбор категории: После нажатия на инлайн-кнопку игры, бот отправляет повторный запрос к БД для извлечения доступных опций именно для этой игры (аккаунты, ключи, валюта) и снова формирует инлайн-меню.
4. Мгновенный парсинг цен: Как только пользователь определяется с конечной категорией, управление передается модулю parser.py. Скрипт переходит по сформированной ссылке на FunPay, собирает верхние блоки объявлений (которые по умолчанию являются самыми релевантными или дешевыми), отсекает лишнее и формирует список из топ-5 позиций. Бот отправляет этот список пользователю в виде структурированного сообщения с ценой, описанием и гиперссылкой для покупки.

4. ТЕСТИРОВАНИЕ И НАДЕЖНОСТЬ СИСТЕМЫ

В процессе тестирования программного комплекса были проверены критические узлы системы:

- Устойчивость к неверному вводу: При вводе несуществующей игры (например, набор случайных символов) бот корректно обрабатывает пустой ответ от БД и отправляет пользователю уведомление: *«Игра не найдена, попробуйте еще раз»*.
 - Конкурентные запросы: Проведено стресс-тестирование одновременной отправки запросов от нескольких пользователей. Благодаря асинхронности фреймворка aiogram 3.x, задержек в генерации интерфейса и обработке кнопок обнаружено не было.
 - Изоляция конфигурации: Проверена безопасность системы — файл .env внесен в .gitignore и не попадает в открытый доступ, защищая данные администратора бота.
-

5. ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта был успешно спроектирован и реализован программный комплекс на языке Python. В процессе работы над проектом были закреплены навыки создания асинхронных приложений, проектирования локальных реляционных баз данных SQLite3 и эффективного извлечения данных методами веб-скрейпинга при помощи библиотеки BeautifulSoup4.

Созданная архитектура разделения логики на отдельные модули (`main_parser.py`, `parser.py`, `bd_games.py`, `titile_game.py`) делает проект легко масштабируемым — в будущем к нему можно без труда подключить другие торговые площадки или расширить аналитический функционал (например, построение графиков изменения цен). Программный продукт полностью выполняет поставленные задачи и готов к эксплуатации.
